

Bloomberg API, Python and VS Code

Follow these steps to reproduce the exact development setup used for the project: the Bloomberg Terminal Desktop API, Python 3.13 with the same packages, and VS Code configured to use them. At the end, a check script confirms that your machine matches.

Component	Version on the reference laptop	Where it lives
Windows	Windows 11 Education, 64-bit	-
Bloomberg Terminal	Terminal installer 2.0, Desktop API (bbcomm)	C:\blp
Python	3.13.16, 64-bit, per-user install	%LOCALAPPDATA%\Programs\Python\Python313
pip	26.2.1 (inside the project environment)	.venv
blpapi (Python SDK)	3.26.9.1, from Bloomberg's package index	.venv
VS Code	1.140.0, User installer, 64-bit	%LOCALAPPDATA%\Programs\Microsoft VS Code
Project environment	Virtual environment named .venv in the project folder	<project>\.venv

Contents

Step	What you do	Time
1	Install and check the Bloomberg Terminal and its API	10 min
2	Install Python 3.13 (64-bit)	5 min
3	Install VS Code and its Python extensions	5 min
4	Get the project code from GitHub	2 min
5	Create the Python environment and install the packages	10 min
6	Connect VS Code to the environment	3 min
7	Verify the whole setup	3 min
8	Set up your account for the shared data collector	5 min
A-C	Daily use, troubleshooting, exact package versions	reference

BEFORE YOU START

What you need

- A Windows 10 or 11 PC, 64-bit. The Bloomberg Desktop API exists only on Windows.
- **Your own** Bloomberg Terminal login. On a library or lab PC the Terminal is usually already installed.
- Internet access, and about 45 minutes. Administrator rights are **not** needed: everything installs for your Windows user only.
- Access to the team's GitHub repository (code) and to the shared OneDrive folder (data).

Important. The Python code can only reach Bloomberg on the **same PC** where the Terminal is running and you are logged in. A laptop without the Terminal can run the analysis on data already downloaded, but cannot download new data.

Note. Type the commands in this guide exactly as shown, in the VS Code terminal or PowerShell. Replace `<you>` with your Windows user name wherever it appears in a path.

STEP 1

Bloomberg Terminal and the Desktop API

Python talks to Bloomberg through a small background program called `bbcomm`, which the Terminal starts automatically. It listens on your PC at port 8194. If the Terminal works and you are logged in, the API normally works too.

1.1 Install the Terminal (skip on lab PCs where it is already installed)

- 1 Go to **[bloomberg.com/professional/support/software-updates](https://www.bloomberg.com/professional/support/software-updates)** and download **Bloomberg Terminal - New Installation**.
- 2 Run the installer and keep the default location `C:\b1p`. Every Python setup in this guide expects it there.
- 3 Restart the PC if the installer asks you to.

1.2 Log in and keep the Terminal open

- 1 Start the Terminal and log in with **your own** username, password and B-Unit or mobile authentication.
- 2 Leave it logged in whenever you run Python scripts. If you log out, or log in on another PC, the scripts lose their connection.

1.3 Check that the API is running

- Open Task Manager (Ctrl+Shift+Esc), tab **Details**, and look for `bbcomm.exe`. On the reference laptop it runs from `C:\b1p\DAPI\bbcomm.exe`.
- In the Terminal you can type `DAPI` then press **GO** (Enter) for Bloomberg's own API help page.
- To check the port, open PowerShell (Windows key, type PowerShell) and run:

```
Test-NetConnection 127.0.0.1 -Port 8194 -InformationLevel Quiet
```

It should print `True`. If it prints `False`, see Appendix B, problem 6.

Note. Bloomberg enforces daily and monthly download limits **per login**. Each teammate has their own allowance, which is why the project collector can share the work between several accounts (Step 8).

STEP 2

Install Python 3.13 (64-bit)

The project uses Python 3.13. Other versions may fail to install some packages or behave differently, so install 3.13 even if you already have another Python.

2.1 Download

- 1 Go to python.org/downloads/windows.
- 2 Under the latest **Python 3.13.x** release, choose **Windows installer (64-bit)**. Do not choose 32-bit or ARM64: blpapi only works with 64-bit Intel/AMD Python.

2.2 Run the installer with these options

- 1 On the first screen, tick **Add python.exe to PATH**.
- 2 Leave **Use admin privileges when installing py.exe** unticked (no admin rights needed).
- 3 Click **Install Now**. Python goes to `C:\Users\<you>\AppData\Local\Programs\Python\Python313`, like on the reference laptop.
- 4 At the end, if offered, click **Disable path length limit**.

2.3 Check the installation

Close any open PowerShell window, open a new one, and run:

```
py -0p
python --version
```

`py -0p` lists the Pythons installed. You should see a line starting with `-V:3.13`. `python --version` should print `Python 3.13.x`.

Important. If `python` opens the Microsoft Store instead, Windows' shortcut is in the way. Open **Settings > Apps > Advanced app settings > App execution aliases** and switch off `python.exe` and `python3.exe`. Then open a new PowerShell window.

STEP 3

Install VS Code and the Python extensions

VS Code is the editor the team uses. Four Microsoft extensions give it Python support.

3.1 Install VS Code

- 1 Go to code.visualstudio.com and download the Windows **User Installer, x64** (version 1.140 or newer).
- 2 During installation, keep **Add to PATH** ticked, and tick **Add 'Open with Code' action** for files and folders.

3 Finish, then start VS Code.

3.2 Install the extensions

Open the Extensions panel (Ctrl+Shift+X), search for each name below and click **Install**. Or install all of them at once from a **new** PowerShell window:

```
code --install-extension ms-python.python
code --install-extension ms-python.vscode-pylance
code --install-extension ms-python.debugpy
code --install-extension ms-python.vscode-python-envs
```

Extension	Identifier	Version on reference laptop	Needed?
Python	ms-python.python	2026.6.0	Required
Pylance	ms-python.vscode-pylance	2026.4.1	Required
Python Debugger	ms-python.debugpy	2026.6.0	Required
Python Environments	ms-python.vscode-python-envs	1.38.0	Required
Claude Code	anthropic.claude-code	2.1.289	Optional (AI assistant)
ChatGPT / Codex	openai.chatgpt, openai.codex-audio	26.930.51102	Optional (AI assistant)

Check. Run `code --list-extensions` in PowerShell. The four `ms-python` entries should be listed.

STEP 4

Get the project code

The code lives in the team's GitHub repository. Downloaded data is never in the repository: it stays in the shared data folder (Step 8).

If Git is installed (git-scm.com, default options), open PowerShell and run:

```
cd $HOME\Documents
git clone https://github.com/tejoker/AFIB-M2.git
```

- 1 Without Git: on the repository page click **Code > Download ZIP** and extract it into `C:\Users\<you>\Documents`. Git is better because `git pull` gets the team's updates.
- 2 Keep the path simple, without special characters or spaces.
- 3 **Do not copy someone else's .venv folder.** A virtual environment only works on the PC where it was created. You create your own in Step 5.

The repository is organised like this:

File or folder	What it is
requirements.txt	Python packages and exact versions
requirements-bloomberg.txt	blpapi, installed from Bloomberg's own package index
verify_setup.py	Checks your setup (Step 7)
.vscode\settings.json	Tells VS Code to use the project's .venv automatically
bloomberg\	Shared Bloomberg collector bbg_collect.py (Step 8), quote example, CME data probe
research\credit_ratings\	IG/HY crossover screen and regression data checks
research\russell_liquidity\	Russell membership and liquidity research
cQG\	CQG Web API connection tests (credentials template included)
cme_challenge\	CME University Trading Challenge research (commodity history)
tests\	Offline tests, no Bloomberg access needed
tools\ , vendor\ , docs\	Program runner, CQG protocol code, this guide
data\	Not in the repository: downloaded data, collector progress, team account list

STEP 5

Create the Python environment and install the packages

Every package is installed in a virtual environment, a .venv folder inside the project. This keeps the project's exact versions separate from anything else on your PC.

5.1 Open the project in VS Code

- 1 In VS Code: **File > Open Folder...** and pick the project folder.
- 2 If VS Code asks **Do you trust the authors of the files in this folder?**, click **Yes, I trust the authors**.
- 3 Open the built-in terminal with **Ctrl+`** (the key left of 1) or **Terminal > New Terminal**. It opens PowerShell in the project folder.

5.2 Create the environment

```
py -3.13 -m venv .venv
```

This creates a .venv folder using Python 3.13. It takes a few seconds and prints nothing.

5.3 Install the packages

Run these three commands one after the other:

```
.venv\Scripts\python.exe -m pip install --upgrade pip
.venv\Scripts\python.exe -m pip install -r requirements.txt
.venv\Scripts\python.exe -m pip install -r requirements-bloomberg.txt
```

The second command installs pandas, numpy, pyarrow and the other packages at the team's exact versions, plus everything they depend on. The third installs `blpapi 3.26.9.1`. Each ends with `Successfully installed ....` Together they take 2 to 5 minutes.

Note. `blpapi` is **not** on the normal Python package site (PyPI), which is why it has its own file: `requirements-bloomberg.txt` points pip to Bloomberg's own package index.

The `blpapi` package already includes the Bloomberg C++ library, so there is nothing else to download from Bloomberg's developer site.

5.4 Allow PowerShell to activate the environment (once per PC)

VS Code activates the environment for you in new terminals. On many PCs, Windows blocks the activation script with the error *running scripts is disabled on this system*. To allow it for your user only, run this once and answer **Y**:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Then close the terminal and open a new one (Ctrl+`). The prompt should now start with `(.venv)`. If your PC does not allow this change, it doesn't matter: always run Python as `.venv\Scripts\python.exe`, as in this guide.

STEP 6

Connect VS Code to the environment

VS Code needs to know it should use the project's `.venv`, both to run scripts and for code completion. The project folder already contains the setting.

The file `.vscode\settings.json` in the project folder contains exactly this:

```
{
  "python.defaultInterpreterPath": "${workspaceFolder}\\venv\\Scripts\\python.exe",
  "python.terminal.activateEnvironment": true
}
```

- 1 If the file is missing, create the folder `.vscode` in the project folder, and in it a file `settings.json` with the content above.
- 2 Press **Ctrl+Shift+P**, type **Python: Select Interpreter** and choose the entry showing `.venv` and Python 3.13.16 (or your 3.13.x).
- 3 Check the bottom-right corner of VS Code: when a `.py` file is open, it should show `3.13.x (.venv)`.
- 4 Open `verify_setup.py` and click the **Run** (triangle) button at the top right. The terminal should list the checks from Step 7.

STEP 7

Verify the whole setup

verify_setup.py checks Python, the packages and the connection to Bloomberg in one go. Run it with the Terminal open and logged in.

```
.venv\Scripts\python.exe verify_setup.py
```

Expected result:

```
[OK] Python version - 3.13.16 (expected 3.13.x)
[OK] 64-bit Python
[OK] Running from the project .venv - ...\\AFIB-M2\\.venv\\Scripts\\python.exe
[OK] package numpy - 2.5.3
[OK] package pandas - 3.0.6
[OK] package pyarrow - 25.0.1
[OK] package openpyxl - 3.1.5
[OK] package XlsxWriter - 3.2.9
[OK] package matplotlib - 3.11.2
[OK] package protobuf - 7.36.2
[OK] package websockets - 17.2
[OK] package yfinance - 1.7.0
[OK] package blpapi - 3.26.9.1
[OK] import blpapi
[OK] Bloomberg API port 8194 (bbcomm)
[OK] Bloomberg session start
[OK] Service //blp/refdata
```

ALL CHECKS PASSED

By default the check only connects to Bloomberg and downloads nothing, so it does not use your daily allowance. Once, to prove that data really comes through, run it with **--data**. That downloads a single value (the S&P 500 last price):

```
.venv\Scripts\python.exe verify_setup.py --data
```

Check. If every line says **[OK]**, your setup matches the reference laptop, down to each package version. Any **[FAIL]** line points to the problem: see Appendix B.

The offline tests check the project code itself, without Bloomberg. They take about a minute:

```
.venv\Scripts\python.exe -m unittest discover -s tests
```

STEP 8

Set up your account for the shared data collector

bloomberg\bbg_collect.py downloads the project dataset. Several teammates can run it, each with their own Bloomberg login: the work is split between accounts and nothing is downloaded twice.

8.1 Connect to the shared data folder

All accounts must write to the **same** data folder, so they can see each other's progress. The team keeps it in a shared OneDrive/SharePoint folder (never on GitHub). Once that folder is synced to your PC, tell the scripts where it is (adapt the path), then close and reopen VS Code:

```
[Environment]::SetEnvironmentVariable("BBG_DATA_DIR",  
    "C:\Users\<you>\OneDrive - <University>\AFIB-data", "User")
```

Without this setting the scripts use the [data](#) folder inside the repository, which only you see.

8.2 Find your account name

The collector identifies you by your Windows user name. In the VS Code terminal run:

```
$env:USERNAME
```

8.3 Add it to the team account list (once, for the whole team)

The file [accounts.json](#) in the shared data folder lists every teammate's account, for example:

```
{"accounts": ["alice", "bob", "carol"]}
```

Important. Agree on the list once, before anyone collects, and **never change the order afterwards**: the order decides which part of the work each account owns. If several people use PCs with the same Windows user name (shared lab logins), add `--account yourname` to every collector command instead.

8.4 Check progress, then run

```
.venv\Scripts\python.exe bloomberg\bbg_collect.py --status  
.venv\Scripts\python.exe bloomberg\bbg_collect.py
```

`--status` shows progress per dataset and per account, without using Bloomberg. The second command collects whatever is still missing, starting with the data the regression needs. It saves after every batch, so you can stop it at any time (Ctrl+C) and run it again later.

- When Bloomberg's daily limit for your login is reached, the collector stops by itself with [Daily capacity reached](#) and refuses to run for 12 hours. That's normal: run it again the next day, while the other accounts carry on.
- If the data folder cannot be shared, each person collects into their own [data](#) folder and you copy them together later: the files never overwrite each other, but some batches may be fetched twice.

Important. Bloomberg licenses its data to each individual user. Keep downloaded data private to the team (never in a public repository or website), and check with whoever manages your Terminals that pooling several logins for one dataset is allowed.

APPENDIX A

Daily use cheat sheet

Run from the VS Code terminal in the project folder, with the Terminal logged in.

Task	Command
Check the setup	<code>.venv\Scripts\python.exe verify_setup.py</code>
Get the team's latest code	<code>git pull</code>
Collector progress, all accounts	<code>.venv\Scripts\python.exe bloomberg\bbg_collect.py --status</code>
Continue collecting	<code>.venv\Scripts\python.exe bloomberg\bbg_collect.py</code>
Collect only what the regression needs	<code>.venv\Scripts\python.exe bloomberg\bbg_collect.py --mandatory-only</code>
Small test run (50 requests)	<code>.venv\Scripts\python.exe bloomberg\bbg_collect.py --max-requests 50</code>
Rebuild the data summary.xlsx	<code>.venv\Scripts\python.exe bloomberg\bbg_collect.py --summary-only</code>
Check data for the regression	<code>.venv\Scripts\python.exe research\credit_ratings\check_regression_data.py</code>
Simple quote example	<code>.venv\Scripts\python.exe bloomberg\query.py</code>
Run the offline tests	<code>.venv\Scripts\python.exe -m unittest discover -s tests</code>
Add a package (tell the team)	<code>.venv\Scripts\python.exe -m pip install name</code> then add it to requirements.txt

Loading collected data in your own scripts

```
import sys
sys.path.insert(0, "bloomberg")          # run from the repository root
from bbg_collect import load_dataset, latest_snapshot

ratings = latest_snapshot()              # latest ratings + reference data per company
annual = load_dataset("fundamentals_annual") # one row per company and fiscal year
prices = load_dataset("prices_daily", columns=["ticker", "date", "PX_LAST"])
```

Troubleshooting

Find the message or symptom you see in the left column.

#	Symptom	Cause and fix
1	<code>python</code> opens the Microsoft Store, or prints nothing	Windows app alias. Settings > Apps > Advanced app settings > App execution aliases: switch off <code>python.exe</code> and <code>python3.exe</code> . Open a new terminal.
2	<code>py</code> is not recognized	Python was installed without the launcher. Rerun the Python installer, choose Modify, tick <code>py</code> launcher. Or use the full path: <code>%LOCALAPPDATA%\Programs\Python\Python313\python.exe -m venv .venv</code>
3	<code>py -3.13: No suitable Python runtime found</code>	Python 3.13 isn't installed (you may have another version). Install 3.13 64-bit (Step 2).
4	<code>No matching distribution found for blpapi</code>	<code>blpapi</code> is only on Bloomberg's index: install it with <code>requirements-bloomberg.txt</code> (Step 5.3), not <code>requirements.txt</code> . On restricted networks, try another network (e.g. eduroam or a phone hotspot).
5	<code>ImportError: DLL load failed</code> when importing <code>blpapi</code>	32-bit or ARM Python. Uninstall it, install Python 3.13 64-bit , delete the <code>.venv</code> folder and redo Step 5.
6	<code>verify_setup: [FAIL] Bloomberg API port 8194</code>	Terminal not started or not logged in, or <code>bbcomm</code> stopped. Log in to the Terminal; if it still fails, close the Terminal completely, run <code>C:\blp\DAPI\bbcomm.exe</code> , then start the Terminal again.
7	<code>[FAIL] Bloomberg session start</code> with the port OK	Login not finished (B-Unit/2FA pending) or the Terminal is logged in on another PC. Finish logging in on this PC and try again.
8	<code>Daily capacity reached</code> or <code>Monthly capacity reached</code>	Your login's Bloomberg download limit. Nothing is broken: wait until the next day (monthly: the next month). The collector saves progress and resumes by itself.
9	<code>running scripts is disabled on this system</code>	PowerShell's execution policy. Run the <code>Set-ExecutionPolicy</code> command in Step 5.4, or ignore it and call <code>.venv\Scripts\python.exe</code> directly.
10	VS Code runs the wrong Python, or underlines <code>"import blpapi"</code> in yellow	<code>Ctrl+Shift+P</code> > Python: Select Interpreter > choose the <code>.venv</code> entry. Check <code>.vscode\settings.json</code> exists (Step 6).
11	<code>code</code> is not recognized in PowerShell	VS Code was installed without Add to PATH. Reinstall with it ticked, or install the extensions from the Extensions panel instead.
12	<code>Account 'x' is not listed in accounts.json</code>	Add your account name (Step 8.2) to <code>accounts.json</code> in the shared data folder, or pass <code>--account</code> with a listed name. If the file only lists you, <code>BBG_DATA_DIR</code> is probably not set (Step 8.1).
13	No module named 'WebAPI' or 'bbg_collect'	Run scripts from the repository root, as written in this guide (e.g. <code>bloomberg\bbg_collect.py</code>), and after a git pull, not from an old copy of the folder.
14	<code>pip</code> fails building a package (error mentions a compiler or wheel)	Usually a Python version other than 3.13, for which no ready-made package exists. Use Python 3.13 and recreate <code>.venv</code> .

Starting over cleanly

If the environment gets into a bad state, delete it and recreate it. This doesn't touch your code or data:

```
Remove-Item -Recurse -Force .venv  
py -3.13 -m venv .venv  
.venv\Scripts\python.exe -m pip install --upgrade pip  
.venv\Scripts\python.exe -m pip install -r requirements.txt  
.venv\Scripts\python.exe -m pip install -r requirements-bloomberg.txt
```

APPENDIX C

Exact package versions

These are the packages the project uses directly, pinned in [requirements.txt](#) and [requirements-bloomberg.txt](#). pip installs exactly these versions and picks their own dependencies automatically; [verify_setup.py](#) checks every version below.

Package	Version	Package	Version
numpy	2.5.3	matplotlib	3.11.2
pandas	3.0.6	protobuf	7.36.2
pyarrow	25.0.1	websockets	17.2
openpyxl	3.1.5	yfinance	1.7.0
XlsxWriter	3.2.9	blpapi	3.26.9.1

Other software on the reference laptop

Software	Version	Notes
Git	2.55.0	Recommended: used to get the code and the team's updates (Step 4).
reportlab	any	Only to rebuild this guide (docs\make_setup_guide.py). Not needed by the project.
Windows PowerShell	5.1	Built into Windows. The commands in this guide are written for it.
Python 3.13 free-threaded build	3.13.16	Installed as an extra option of the Python installer. Not used by the project.